

Software Engineering

Lab 3: Component Modelling & Architectural Pattern Selection

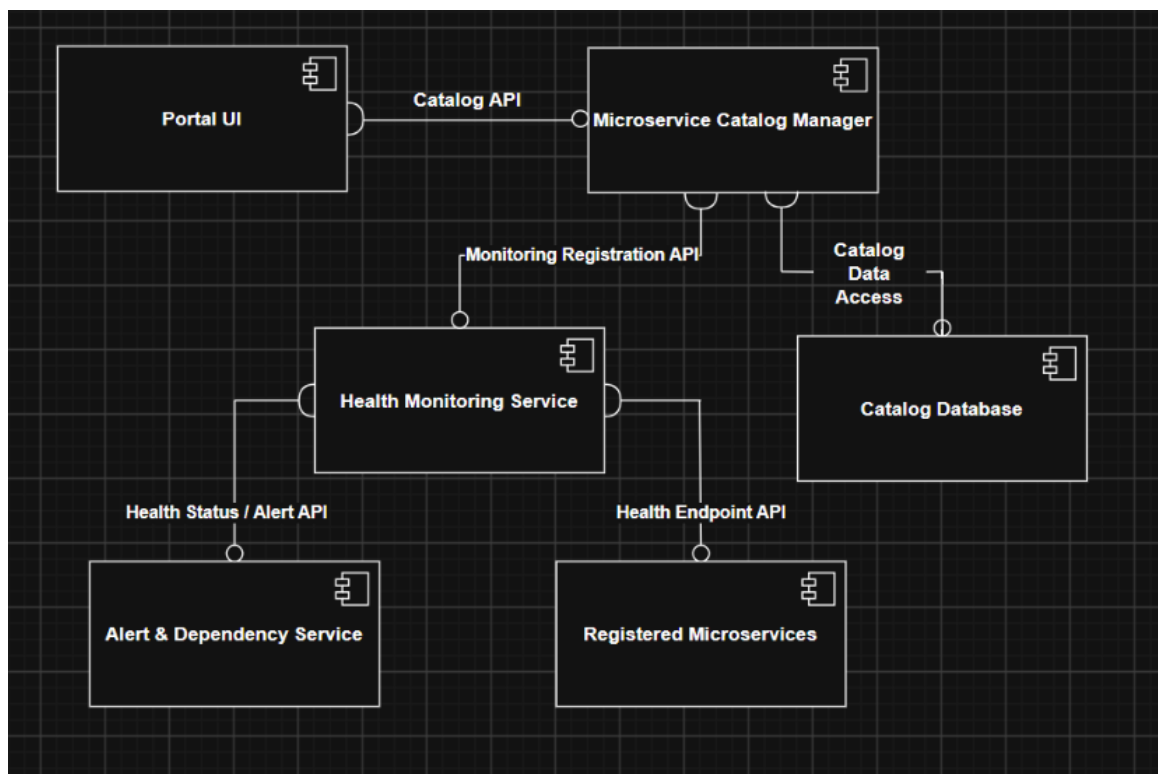
PROBLEM STATEMENT #42: Internal Microservice Catalog & Health Portal

Name: Nandhana S

SRN: PES1UG24AM440

Section: H

Component diagram:



Architecture Selection:

Architectural Choice Justification:

The system is divided into independently focused components: Portal UI, Microservice Catalog Manager, Health Monitoring Service, Catalog Database, Alert & Dependency Service,

and the registered microservices being monitored. This structure fits a portal whose main purpose is to manage and monitor multiple microservices.

Reason 1: Independent Services and Fault Isolation

The catalog management, health monitoring, and alert/dependency functions can operate as separate services. A failure or change in one function can therefore be isolated without requiring the whole portal to be redeployed. This is especially useful because the system continuously monitors registered microservices.

Reason 2: Independent Scaling and Deployment

Health monitoring may generate repeated requests because registered services are checked periodically. With a microservices architecture, the Health Monitoring Service can be scaled independently when the number of registered services increases. Similarly, catalog or alert functionality can be updated and deployed independently.

Security Advantage:

Security responsibilities can be isolated at service boundaries. The Portal UI can communicate with the Microservice Catalog Manager through a controlled Catalog API, while health-monitoring and alert operations are exposed only through their required service interfaces. This limits unnecessary direct access between components and makes access control easier to enforce at API boundaries.

Performance Benefit:

The Health Monitoring Service can perform health checks independently and in parallel rather than making the Portal UI wait for every health-check operation. This keeps the user-facing portal responsive while monitoring continues in the background.

Components:

1. Portal UI
2. Microservice Catalog Manager
3. Health Monitoring Service
4. Catalog Database
5. Alert & Dependency Service
6. Registered Microservices

Key Interfaces

- Catalog API : Portal UI requires the API; Microservice Catalog Manager provides it.
- Catalog Data Access :Microservice Catalog Manager requires data access; Catalog Database provides it.
Monitoring Registration API :Microservice Catalog Manager requests monitoring registration; Health Monitoring Service provides it.
- Health Endpoint API :Health Monitoring Service requires health endpoints; Registered Microservices provide them.
- Health Status / Alert API: Health Monitoring Service sends health-status information; Alert & Dependency Service provides alert/dependency handling.

Microservices introduce additional operational complexity, network communication, and possible data-consistency challenges. However, for this problem, the benefits of independent deployment, scaling, and fault isolation are appropriate because the system is specifically designed to manage and monitor multiple microservices.